ABP v10.0 Highlights & Community News



Engincan VESKE

Software Engineer at Volosoft

@EngincanVeske

EngincanV

engincanveske.substack.com

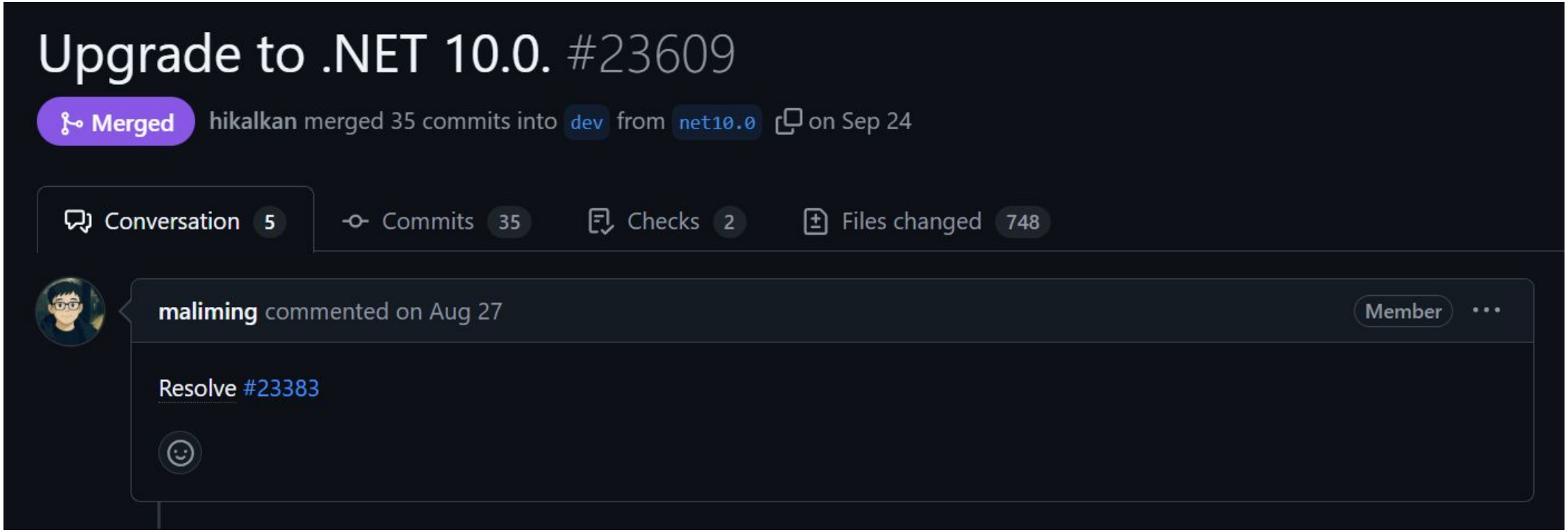
@engincanv

ABP 10.0 Highlights

- Upgraded to .NET 10.0
- Upgraded to Blazorise v1.8.2
- Elsa Workflow Module (PRO)
- New Object Mapper: Mapperly
- Localization: Nested Object Support in JSON Files
- EF Core Shared Entity Types on Repositories
- Angular SSR Support

Read the blog post

Upgraded to .NET 10.0



abp.io

1-) Update global.json files:

```
{
  "sdk": {
-    "version": "9.0.304"
+    "version": "10.0.100"
  }
}
```

2-) Update the target framework:

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
-    <TargetFramework>net9.0</TargetFramework>
+    <TargetFramework>net10.0</TargetFramework>
  </PropertyGroup>

</Project>
```

3-) Update package references:

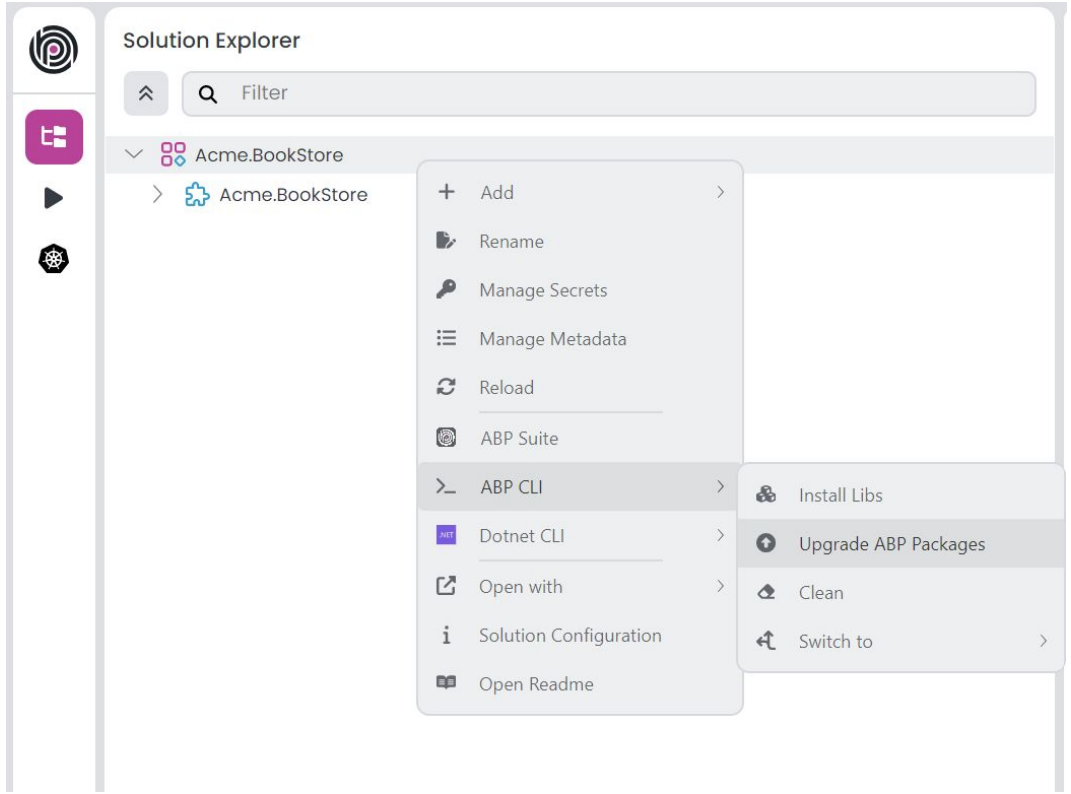
```
<ItemGroup>
-   <PackageReference Include="Microsoft.AspNetCore.JsonPatch" Version="9.0.0" />
-   <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="9.0.0" />
-   <PackageReference Include="Microsoft.Extensions.Caching.Abstractions" Version="9.0.0" />
-   <PackageReference Include="System.Net.Http.Json" Version="9.0.0" />
+   <PackageReference Include="Microsoft.AspNetCore.JsonPatch" Version="10.0.0" />
+   <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="10.0.0" />
+   <PackageReference Include="Microsoft.Extensions.Caching.Abstractions" Version="10.0.0" />
+   <PackageReference Include="System.Net.Http.Json" Version="10.0.0" />
</ItemGroup>
```

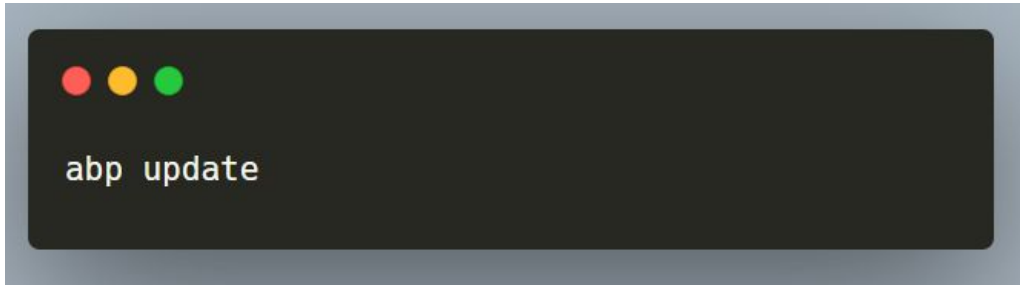
abp.io

4-) Check release notes & migration guides (and apply the required changes):

- Migrate from ASP.NET Core in .NET 9 to ASP.NET Core in .NET 10 Guide
- Breaking Changes in .NET 10 Documentation

5-) Use ABP Studio (or ABP CLI) to update to ABP 10.0 (and check the migration guide for fixes on your side):



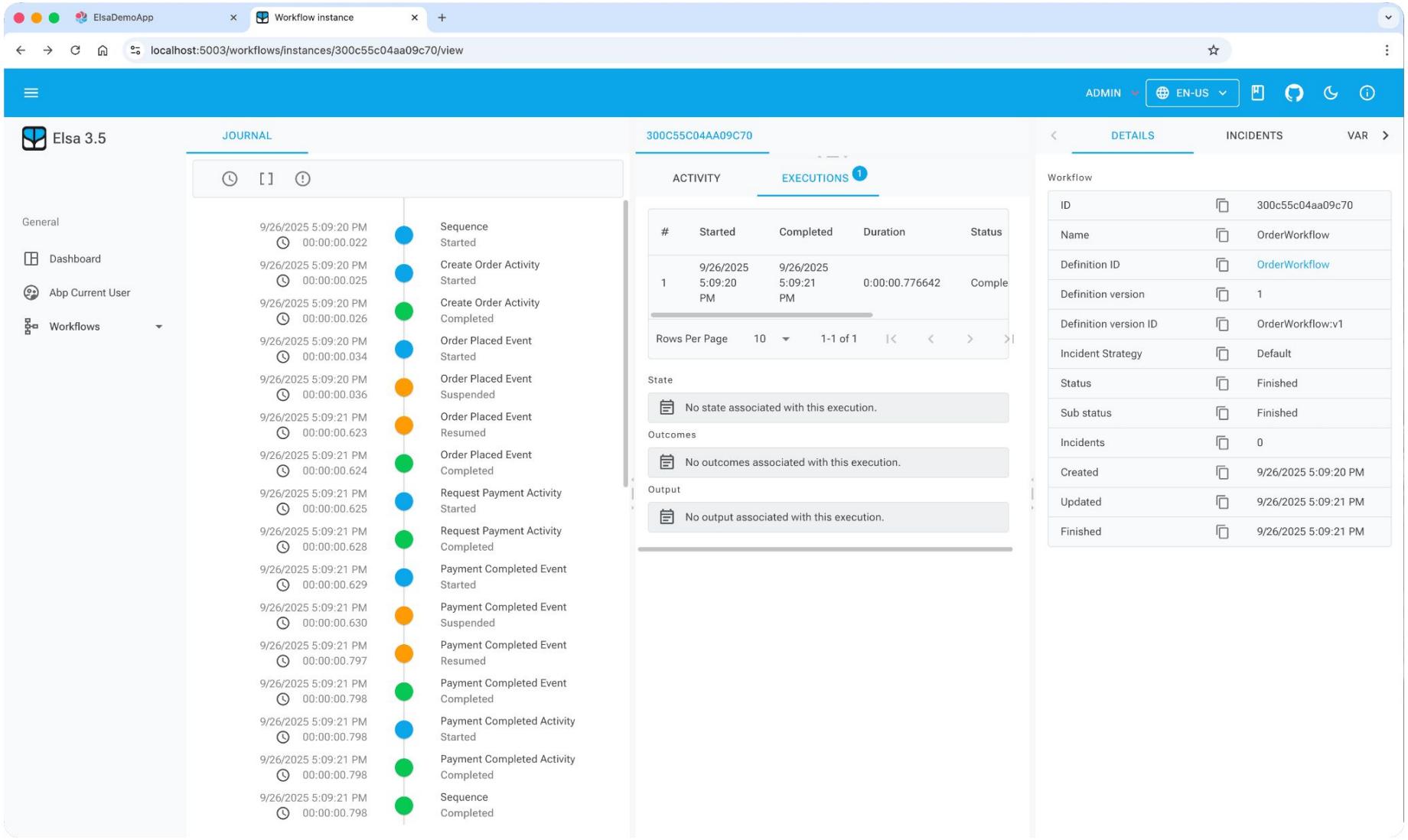


Upgraded to Blazorise v1.8.2

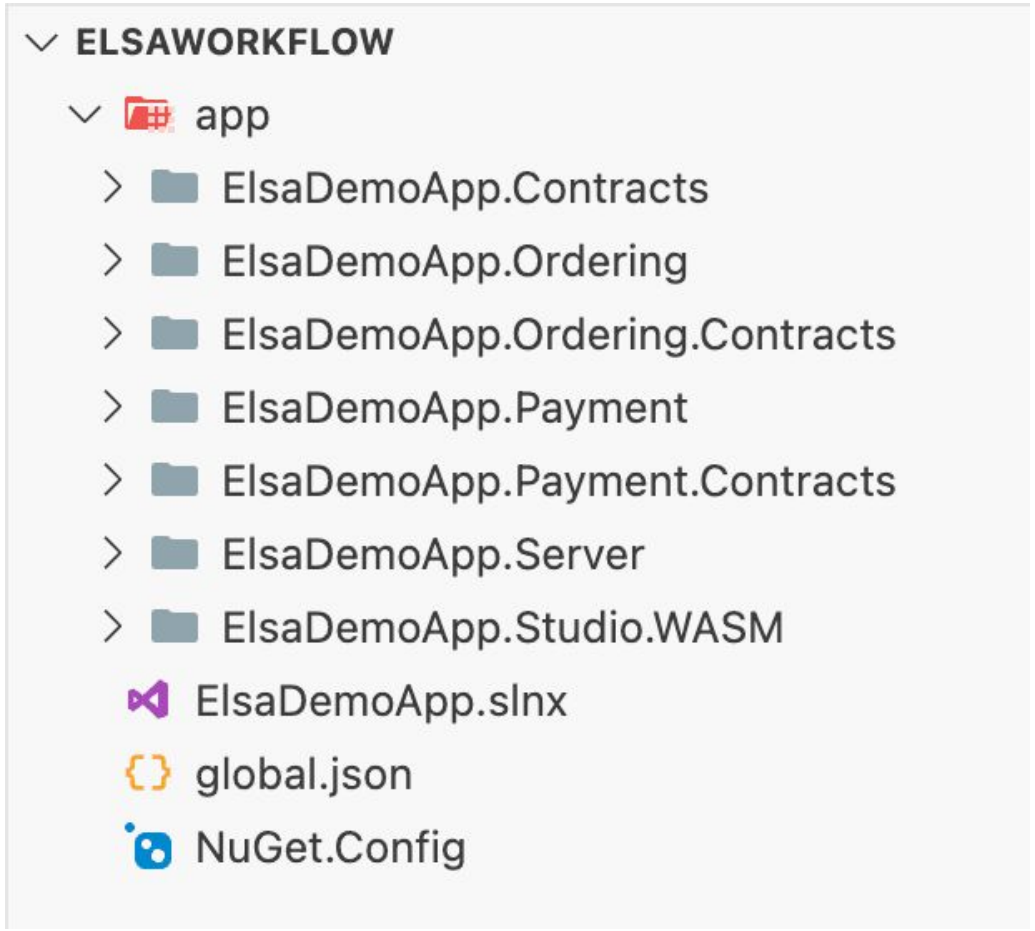
```
22  +     <PackageVersion Include="Blazorise" Version="1.8.2" />
23  +     <PackageVersion Include="Blazorise.Components" Version="1.8.2" />
24  +     <PackageVersion Include="Blazorise.DataGrid" Version="1.8.2" />
25  +     <PackageVersion Include="Blazorise.Snackbar" Version="1.8.2" />
```

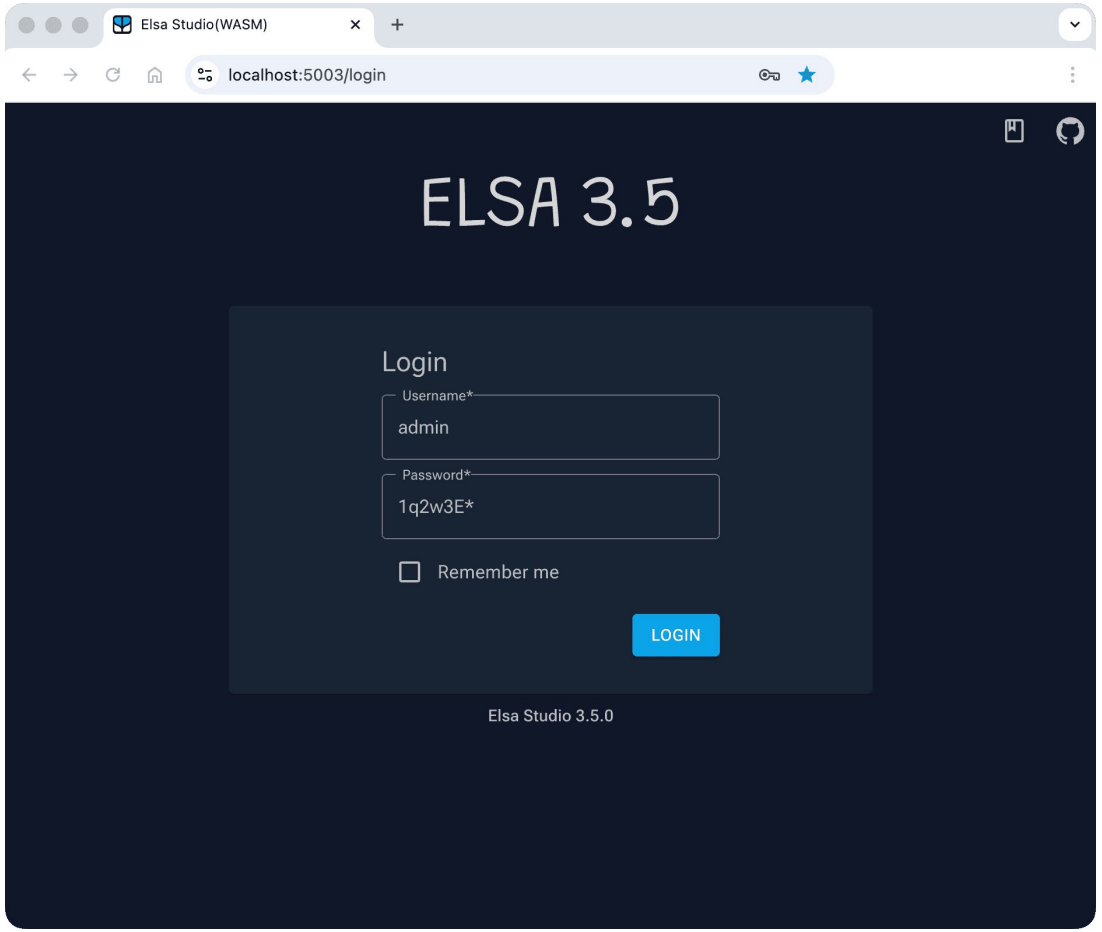
abp.io

Elsa Workflow Module (PRO)



abp.io





Sample: https://abp.io/docs/10.0/samples/elsa-workflows-demo

Documentation: https://abp.io/docs/10.0/modules/elsa-pro

abp.io

New Object Mapper: Mapperly

AutoMapper and MediatR Going Commercial



Jimmy Bogard

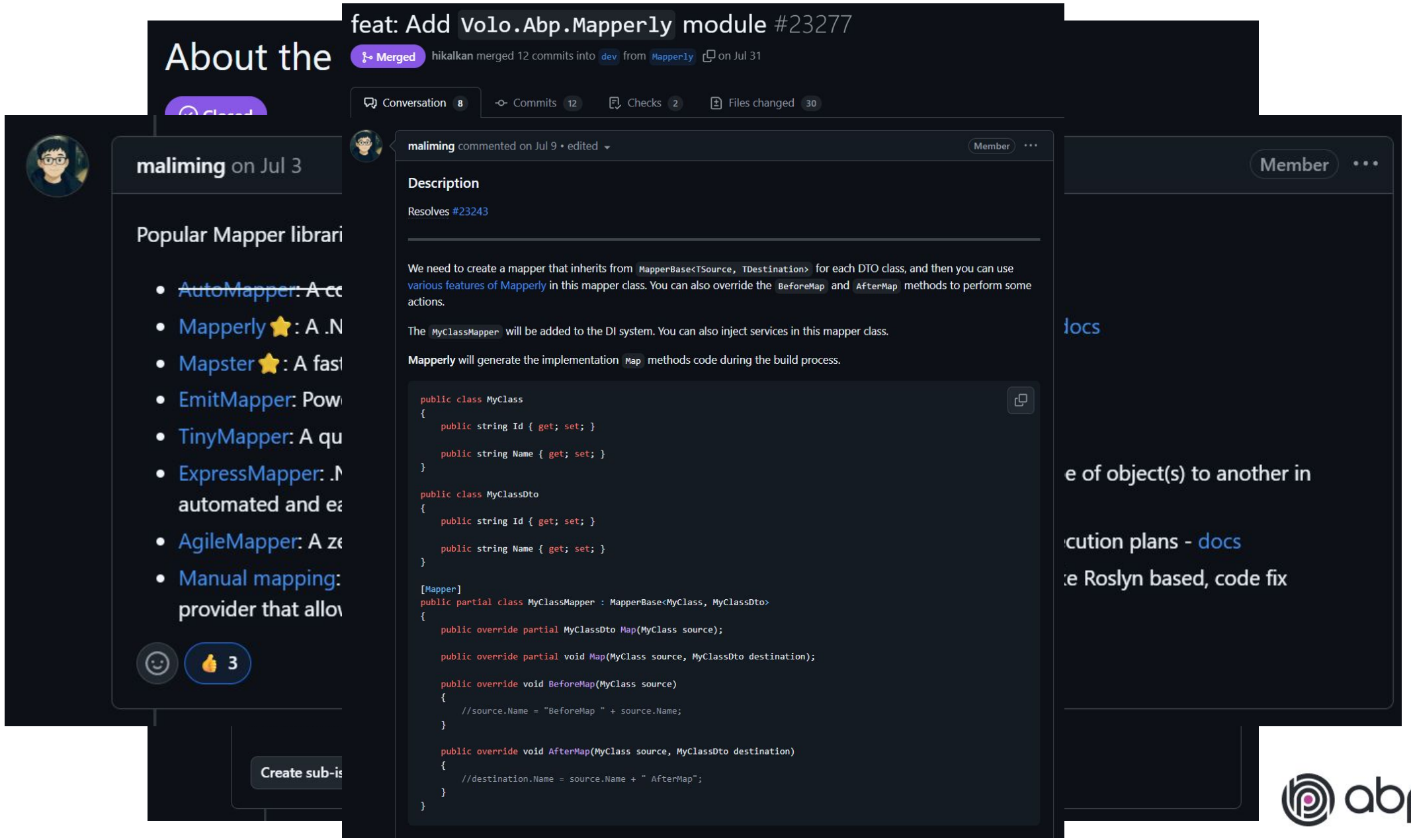
Apr 2, 2025 • 3 min read

Yes, another one of "those posts". But tl;dr:

In order to ensure the long-term sustainability of my OSS projects, I will be commercializing AutoMapper and MediatR.

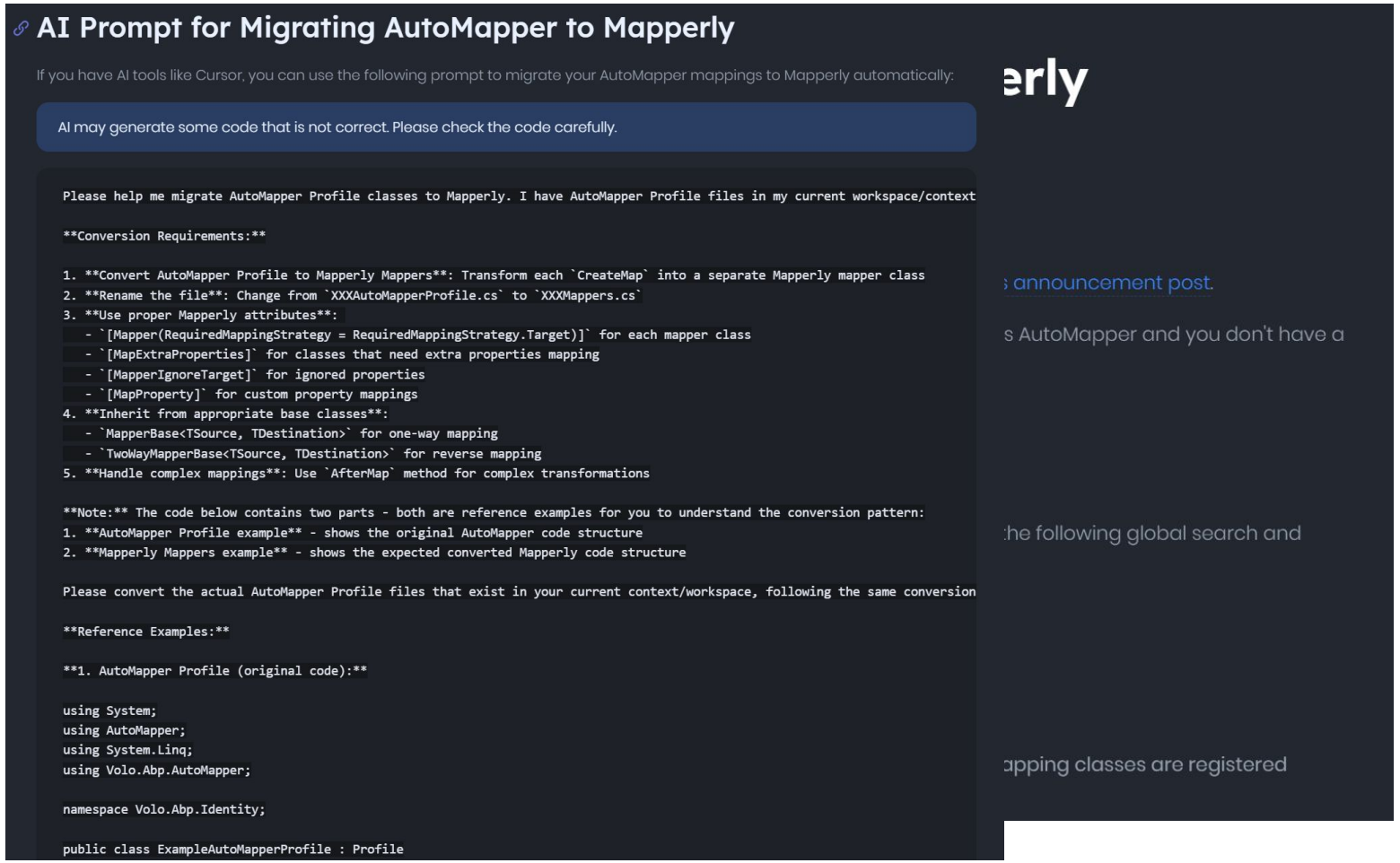
I did not post this on April 1st for obvious reasons. But first a little background on how I got to this point.

abp.io


About the
feat: Add Volo.Abp.Mapperly module #23277
Merged
hikalkan merged 12 commits into dev from Mapperly on Jul 31
Conversation 8
Commits 12
Checks 2
Files changed 30
maliming on Jul 3
maliming commented on Jul 9 • edited
Member
Description
Resolves #23243
We need to create a mapper that inherits from MapperBase<TSource, TDestination> for each DTO class, and then you can use various features of Mapperly in this mapper class. You can also override the BeforeMap and AfterMap methods to perform some actions.
The MyClassMapper will be added to the DI system. You can also inject services in this mapper class.
Mapperly will generate the implementation Map methods code during the build process.
public class MyClass
{
public string Id { get; set; }
public string Name { get; set; }
}
public class MyClassDto
{
public string Id { get; set; }
public string Name { get; set; }
}
[Mapper]
public partial class MyClassMapper : MapperBase<MyClass, MyClassDto>
{
public override partial MyClassDto Map(MyClass source);
public override partial void Map(MyClass source, MyClassDto destination);
public override void BeforeMap(MyClass source)
{
//source.Name = "BeforeMap " + source.Name;
}
public override void AfterMap(MyClass source, MyClassDto destination)
{
//destination.Name = source.Name + " AfterMap";
}
}
Popular Mapper librari
AutoMapper: A cc
Mapperly ⭐: A .N
Mapster ⭐: A fast
EmitMapper: Pow
TinyMapper: A qu
ExpressMapper: .N
automated and ea
AgileMapper: A ze
Manual mapping:
provider that allo
3
Member
docs
e of object(s) to another in
cution plans - docs
e Roslyn based, code fix
Create sub-is

abp.io

⚠️Volo.Abp.AutoMapper package remains available for **backward compatibility**. You can keep using **AutoMapper** in your solutions, but you are responsible for obtaining and managing its license if needed.



AI Prompt for Migrating AutoMapper to Mapperly

If you have AI tools like Cursor, you can use the following prompt to migrate your AutoMapper mappings to Mapperly automatically:

AI may generate some code that is not correct. Please check the code carefully.

```
Please help me migrate AutoMapper Profile classes to Mapperly. I have AutoMapper Profile files in my current workspace/context

**Conversion Requirements:**

1. **Convert AutoMapper Profile to Mapperly Mappers**: Transform each `CreateMap` into a separate Mapperly mapper class
2. **Rename the file**: Change from `XXXAutoMapperProfile.cs` to `XXXMappers.cs`
3. **Use proper Mapperly attributes**:
   - `[Mapper(RequiredMappingStrategy = RequiredMappingStrategy.Target)]` for each mapper class
   - `[MapExtraProperties]` for classes that need extra properties mapping
   - `[MapperIgnoreTarget]` for ignored properties
   - `[MapProperty]` for custom property mappings
4. **Inherit from appropriate base classes**:
   - `MapperBase<TSource, TDestination>` for one-way mapping
   - `TwoWayMapperBase<TSource, TDestination>` for reverse mapping
5. **Handle complex mappings**: Use `AfterMap` method for complex transformations

**Note:** The code below contains two parts - both are reference examples for you to understand the conversion pattern:
1. **AutoMapper Profile example** - shows the original AutoMapper code structure
2. **Mapperly Mappers example** - shows the expected converted Mapperly code structure

Please convert the actual AutoMapper Profile files that exist in your current context/workspace, following the same conversion

**Reference Examples:**

**1. AutoMapper Profile (original code):**

using System;
using AutoMapper;
using System.Linq;
using Volo.Abp.AutoMapper;

namespace Volo.Abp.Identity;

public class ExampleAutoMapperProfile : Profile
```

erly

announcement post.

s AutoMapper and you don't have a

he following global search and

apping classes are registered

Read the migration guide:

Localization: Nested Object Support in JSON

Declaration (nested objects):

```
{
  "culture": "en",
  "texts": {
    "MyNestedTranslation": {
      "SomeKey": "Some nested value",
      "SomeOtherKey": "Some other nested value"
    }
  }
}
```

Usage:

```
L["MyNestedTranslation__SomeKey"];
L["MyNestedTranslation__SomeOtherKey"];
```

EF Core Shared Entity Types on Repositories

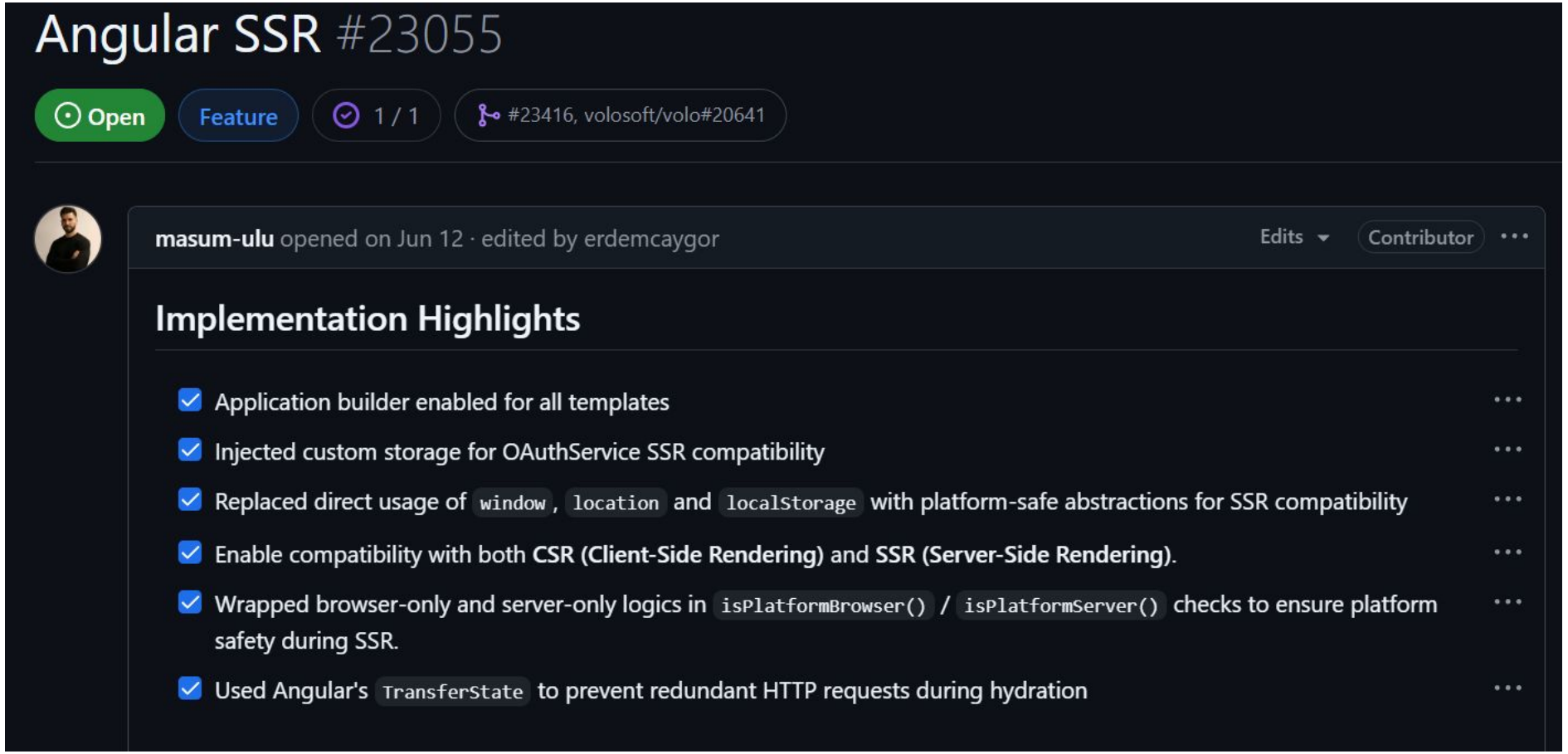
ABP now support EF Core **shared-type entity** types by allowing a custom entity name to be set on a repository before performing operations. Internally, it uses EF Core's **DbContext.Set<T>(string name)** to target the correct **DbSet**/table for the same CLR type, enabling scenarios like per-tenant tables, archives, or partitioning, and you can switch the target at runtime.

```
// Set the shared entity name so repository operations target that table
var repo = serviceProvider.GetRequiredService<IRepository<MyEntity, Guid>>();
repo.SetCustomEntityName("MyEntity_TenantA");
var list = await repo.GetListAsync();

// Switch to another shared name later on the same instance
repo.SetCustomEntityName("MyEntity_Archive");
await repo.InsertAsync(new MyEntity { /* ... */ });
```

abp.io

Angular SSR



abp.io

Migration Guides (v9.x to v10.0)

- Upgraded to .NET 10.0 (Migrate from .NET 9 to .NET 10)
- Upgraded to OpenIddict 7.X
- Migrated from AutoMapper to Mapperly
- Removed Obsolete Services (RazorRuntimeCompilation and IActionContextAccessor)
- Added BLOB Storing Memory Provider (for testing purposes)

Read the migration guide

Community News

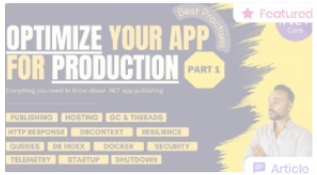
- **Community Posts** (20+ articles & videos)
- **Announcements**
  - Sponsored to .NET Conf 2025!
  - Weekly Webinar: Discover ABP Platform

abp.io

Community Posts

(20+ articles & videos)

https://abp.io/community/



Optimize Your .NET App for Production - Complete Checklist (Part 1)

alper
21 days ago

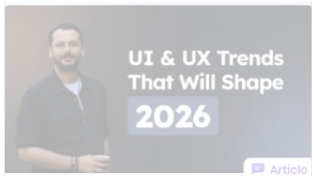
dotnet performance optimize aspnetcore kestrel best-practises



What is That Domain Service in DDD for .NET Developers

salih
2 days ago

domain-service ddd abp-framework dotnet



UI & UX Trends That Will Shape 2026

armagan
7 days ago

application-development best-practices performance optimization accessibility



Return Code vs Exceptions: Which One is Better?

alper
12 days ago

exception-handling best-practices error-handling



5 Things You Should Keep in Mind When Deploying to a Clustered Environment

selmankoc
13 days ago

deployment state-management logging background-worker clustered



A Developer's Guide to Distributed Event Buses in .NET

emre.kara
2 months ago

azure event-bus rabbitmq dotnet distributed-events



Building Dynamic Forms in Angular for Enterprise Applications

erdem.caygor
about a month ago

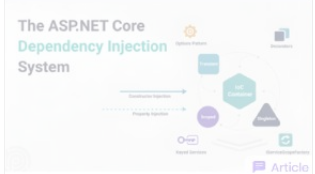
angular customization application-development dynamic-form-extensions



Optimize Your .NET App for Production - Complete Checklist (Part 2)

alper
21 days ago

dotnet performance optimize aspnetcore kestrel best-practises



The ASP.NET Core Dependency Injection System

oguzhan.agir
22 days ago

dependency-injection best-practices aspnet-core aot dotnet-9.0



Uncovering ABP's [illegible] Magic: Supercharging ASP.NET Core Development

berkansasmaz
22 days ago

abp-framework abp abp-features application-development asp.net-core



[illegible] Exception Handling Mistakes in .NET (and How to Actually Fix Them)

[illegible]
26 days ago

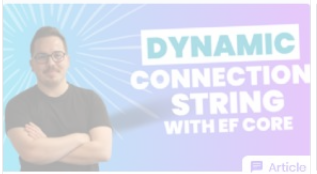
dotnet exception-handling best-practices asp.net



[illegible] Library Linking Made Easy: Paths, Workspaces, and Symlinks

[illegible] kurtulus
26 days ago

angular cli application-development best-practices



How to Dynamically Set the Connection String in EF Core

berkansasmaz
2 months ago

EfCore application-configuration entity-framework-core asp.net-core



Demystified Aggregates in DDD & .NET: From Theory to Practice

mansur.besleney
2 months ago

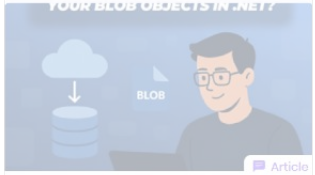
entity-framework-core ddd dotnet



From Server to Browser: Angular TransferState Explained

erdem.caygor
27 days ago

angular state-management prerendering http-api interceptors



Where and How to Store Your BLOB Objects in .NET?

EngincanV
about a month ago

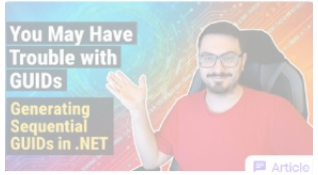
abp-framework blob-storing blob-storage .net



Building Scalable Angular Apps with Reusable UI Components

sumeyye.kurtulus
about a month ago

angular application-development modularity best-practices frontend



You May Have Trouble with GUIDs: Generating Sequential GUIDs in .NET

enisn
about a month ago

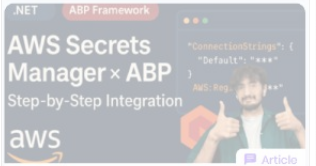
sql entity-framework-core dotnet



Why Do You Need Distributed Locking in ASP.NET Core

fahrigedik
about a month ago

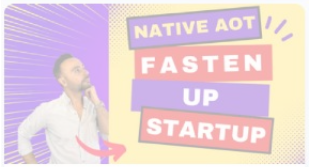
redis concurrency abp-framework distributed-events asp.net-core



Step-by-Step AWS Secrets Manager Integration in ABP Framework Projects

fahrigedik
2 months ago

application-configuration abp-framework abp tutorial best-practices



Native AOT for .NET — How to Fasten Startup Time and Memory Footprint

alper
about a month ago

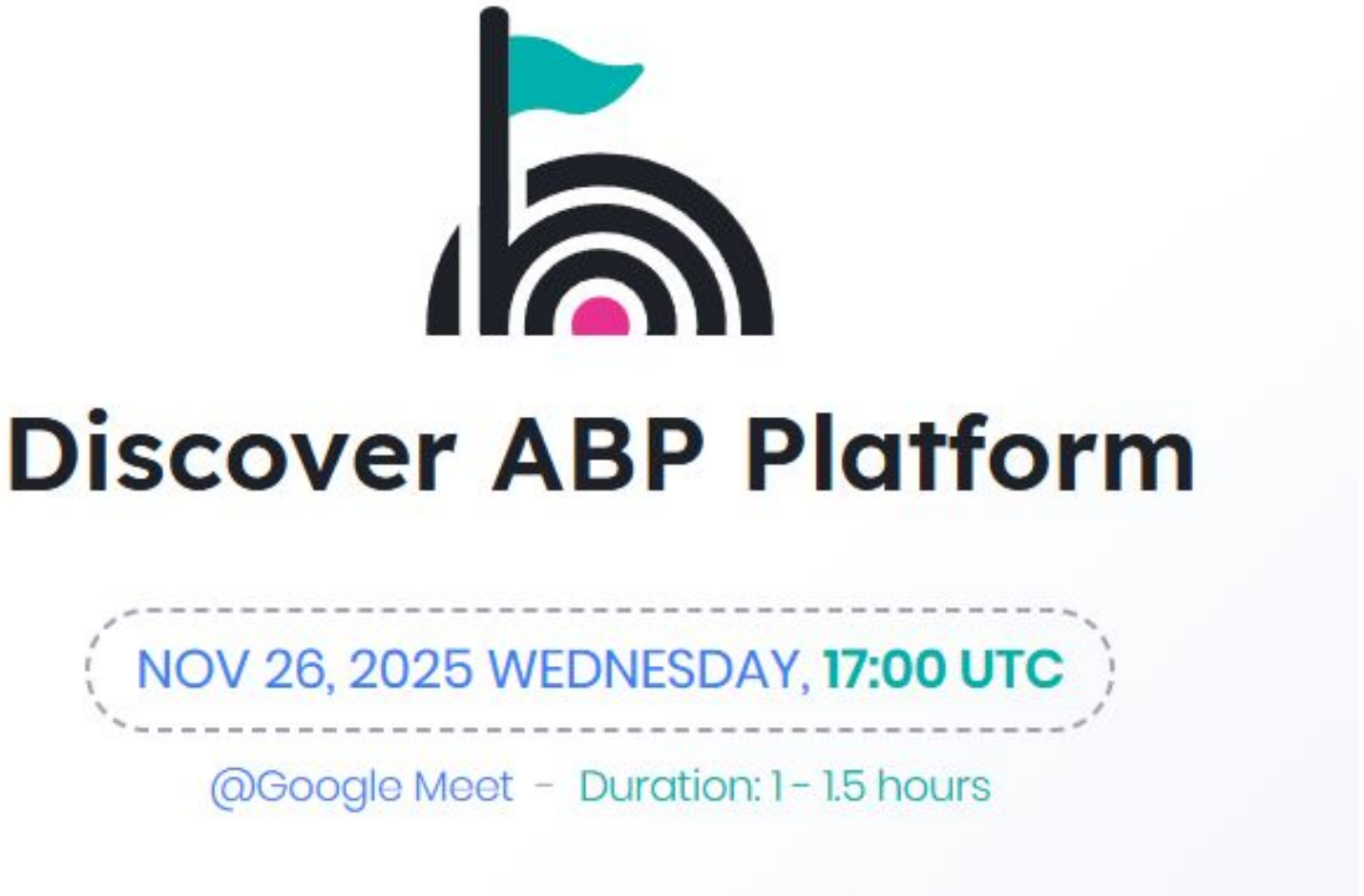
dotnet optimiz

Sponsored to .NET Conf 2025!



abp.io

Weekly Webinar: Discover ABP Platform



For free registration:

Thanks for listening!